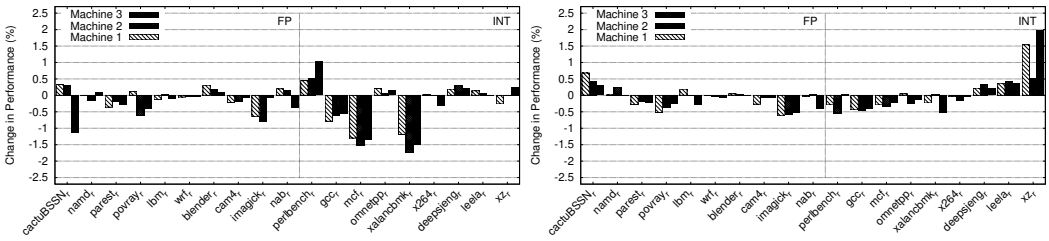

(a) After making LLVM sound (b) Sound prototype + optimizations

Fig. 7. Change in run time of SPEC CPU 2017 for our two prototypes. Higher bars indicate speedup relative to baseline LLVM, lower bars indicate slowdown.

- p and q are both computed by the **gep inbounds** with same base pointer.
- Either p or q is computed by a series of **gep inbounds** with positive offsets, based on the same base pointer.

The number of lines changed in this prototype against the previous one totals about 1274 lines. Overall, our "sound and fast" prototype requires changing less than 1700 lines of code in LLVM to make it sound against our memory model with minor impact on run time performance.

6 PERFORMANCE EVALUATION

For our memory model to have a chance of being adopted, we must establish that it does not interfere with the quality of generated code.

6.1 Setup

We used three machines, with the following CPUs, to measure performance: (1) Intel i3-6100 (Skylake), (2) Intel i5-6600 (Skylake), and (3) Intel i7-7700 (Kaby Lake). All machines have 8 GB RAM and Ubuntu 16.04 installed. To get more consistent results, Intel Hyperthreading, TurboBoost, SpeedStep, and Speed Shift were disabled, and the CPU scaling governor was set to "performance." We also disabled non-essential system services and address space layout randomization.

As benchmarks, we used SPEC CPU 2017 and the LLVM Nightly Test suite (293 C/C++ benchmarks). For SPEC CPU, we used only the SPECrate benchmarks because of RAM limitations (SPECspeed requires more memory than our machines had). The Fortran benchmarks were compiled first with `gfortran`.

Each benchmark was compiled with `-O3`, executed three times, and the median running time is used to compare performance. To run LLVM Nightly Tests, we used the `cpuset` utility to pin the benchmark to a single core. Unlike SPEC, these short-running benchmarks were seeing significant variance due to migrations between cores. Also, we used a RAM disk to minimize fluctuations due to disk access time. Some benchmarks in the LLVM suite have short running time, so using this setting is helpful to stabilize the results.

When comparing performance for LLVM Nightly Tests, we exclude benchmarks where our changes to Clang and LLVM did not result in any changes to the resulting object code. Some benchmarks had high performance variation (> 5%) across runs, most of these were benchmarks that run too quickly for accurate timing. We excluded these from our results as well.

6.2 Performance of Generated Code

Performance after making LLVM sound. Figure 7a shows the effect that our first (sound but unoptimized) prototype had on the performance of SPEC CPU 2017. On average, across all benchmarks and machines, we observed a 0.2% slowdown. The worst slowdown was a 1.75% regression